W4111 Spring 2026 - Exam 3 Study Guide

Open-book reference. Lecture 6 (Normalization only) + Lectures 9, 10, 11, 12 (Module IV / REST V2).

Compiled for dff9

Spring 2026

Contents

How to use this guide

- The exam is **open book / open paper**. The PDF table of contents above lists every subsection and its **page number** so you can jump straight to the topic you need.
- Each section ends with a **"Find it in the slides"** pointer giving the lecture deck and slide range, in case you want to verify against the original lectures.
- Sources used:
  - **Lecture 6** (slides 47-76, normalization only)
  - **Lecture 9** - NoSQL-3 (full deck, 113 slides) - query processing, optimization, MongoDB, Neo4j, REST intro
  - **Lecture 10** - Module II-4 (full deck, 92 slides) - optimization finish, transactions, isolation, CAP, scale, REST
  - **Lecture 11** - Module II-4 (full deck, 90 slides) - serializability, isolation, CAP, big data, Spark, REST
  - **Lecture 12 - V2** - Module IV REST/Web Apps (full deck, 76 slides) - deadlock, OLAP, ETL, MapReduce, Spark, REST
- The Big Data deck (`Lecture-12-Module-II-6-Big-Data.pdf`) is **not** used as a source per scope; everything from it that matters is also in Lecture 11 and the V2 deck.
- "Slide N" always means the slide-deck slide number, which is also the PDF page number for the source decks.

Part 1: Normalization (Lecture 6, slides 47-76)

1.1 Why normalize? Anomalies and redundancy

Definition. Normalization is a process for redesigning relations so that redundant data and the anomalies it causes are eliminated.

- **Redundancy** is when a fact is stored in more than one tuple. Example: storing `(building, budget)` in every `instructor` row that shares a `dept_name`.
- The three **update anomalies** redundancy creates:
  - **Update anomaly** - changing the budget of a department requires updating every instructor row.
  - **Insertion anomaly** - cannot record a new department until you have at least one instructor in it (the FK forces you to invent dummy data).
  - **Deletion anomaly** - deleting the last instructor of a department also deletes the only copy of the department's budget.
- **Goal of normalization:** every non-trivial fact is stored exactly once. The decomposition must be **lossless** (no data invented or lost on natural join) and ideally **dependency-preserving** (every original FD is enforceable on a single decomposed relation).

Find it in the slides: Lecture 6, slides 49-51.

1.2 Functional dependencies

Definition. A functional dependency (FD) `X -> Y` says that for any two tuples that agree on `X`, they must also agree on `Y`. Formally, on every legal instance of `R`: `t1[X] = t2[X]` implies `t1[Y] = t2[Y]`.

- An FD is **trivial** if `Y` is a subset of `X` (e.g., `(A, B) -> A`). Trivial FDs hold automatically.
- An FD is a **generalization of a key**. If `X -> R` (i.e., `X` determines all attributes), then `X` is a **superkey**.
- FDs are constraints declared by the designer; they hold on **all** legal instances, not just the ones currently in the table. You can prove an FD is **violated** by data, but you cannot prove an FD **holds** from data alone.

Sample-question pattern. When a question gives sample tuples and asks "which FDs hold based on the data shown," check each candidate `X -> Y` by grouping rows with the same `X` value and verifying they agree on `Y`. A single counterexample disproves it.

Example. Given the data:

A	B	C
1	x	a
1	x	b
2	y	a

- `A -> B` **holds**: every row with `A=1` has `B=x`; the only row with `A=2` has `B=y`.

- `A -> C` **does NOT hold**: `A=1` appears with both `C=a` and `C=b`. That is a counterexample.
- `C -> A` **does NOT hold**: `C=a` appears with both `A=1` (row 1) and `A=2` (row 3).
- `B -> A` **does** hold here (each `B` value goes with one `A`), but remember: data alone can never *prove* an FD - it can only disprove it. The designer asserts which FDs are required to hold on every legal instance.

Find it in the slides: Lecture 6, slides 56-58.

1.3 Closure (F+) and attribute closure

Definition. The **closure of a set of FDs F**, written `F+`, is every FD that can be logically derived from `F` using **Armstrong's axioms** (reflexivity, augmentation, transitivity).

- **Attribute closure** `X+` (under `F`) is the set of all attributes functionally determined by `X`. It is the standard tool for testing keys and FDs:
  - `X` is a **superkey** iff `X+ = R`.
  - `X` is a **candidate key** iff it is a superkey and no proper subset of it is a superkey.
  - `X -> Y` is in `F+` iff `Y` is a subset of `X+`.

Algorithm to compute `X+`:

```
result := X
repeat:
  for each FD A -> B in F:
    if A is a subset of result, then result := result U B
until result stops changing
```

Worked example. `R(A, B, C, D, E)` with `F = { A -> B, B -> C, CD -> E }`.

Compute `{A, D}+`: start with `{A, D}`. Apply `A -> B` -> `{A, B, D}`. Apply `B -> C` -> `{A, B, C, D}`. Apply `CD -> E` -> `{A, B, C, D, E} = R`. So `{A, D}` is a **superkey**. `{A}+ = {A, B, C}` is not a superkey (`D, E` missing), so `{A}` alone is not a key but `{A, D}` is a candidate key (and minimal, since `{A}+ != R` and `{D}+ = {D} != R`).

Find it in the slides: Lecture 6, slides 59-61.

1.4 Keys: superkey, candidate key, primary key

Term	Definition
Superkey	Any attribute set `X` with `X+ = R`.
Candidate key	Minimal superkey (no proper subset is a superkey).
Primary key	A candidate key chosen by the designer to be the row identifier.
Prime attribute	Any attribute that appears in **some** candidate key.
Non-prime attribute	An attribute that is **not** in any candidate key.

Find it in the slides: Lecture 6, slide 60.

1.5 Decomposition: lossless-join and dependency-preserving

Definition. Decomposition splits a relation `R` into `R1, R2, ...` whose union of attributes is `R`.

- **Lossless-join decomposition** of `R` into `R1, R2`: the natural join `R1 NATURAL JOIN R2 = R` for every legal instance. The standard sufficient test (binary case): the common attributes `R1 INTERSECT R2` form a superkey of at least one of `R1` or `R2`.
- **Dependency-preserving decomposition:** every FD in `F` is enforceable by checking only one of the decomposed relations (no need to compute joins to check constraints).
- A bad decomposition is **lossy**: the natural join produces extra spurious tuples not in the original relation (information is "lost" in the sense that you can no longer reconstruct exactly the original).

Lossy example. `R(A, B, C)` with the data `{(1, x, p), (2, x, q)}`. Decompose into `R1(A, B) = {(1,x), (2,x)}` and `R2(B, C) = {(x,p), (x,q)}`. The natural join on `B` gives `{(1,x,p), (1,x,q), (2,x,p), (2,x,q)}` - two spurious tuples. The split is lossy because `B` (the common attribute) is not a superkey of either piece. Compare to the BCNF zip-code split in 1.6: there `zip` is a key of `R1(zip, city)`, so the join recovers `R` exactly.

Find it in the slides: Lecture 6, slides 53-55.

1.6 Boyce-Codd Normal Form (BCNF)

Definition. `R` is in **BCNF** with respect to `F` iff **for every non-trivial FD `X -> Y` in `F+`, `X` is a superkey of `R`**.

- Equivalently: the only non-trivial FDs allowed are those whose left-hand side is a superkey.
- **BCNF decomposition algorithm.** While `R` is not in BCNF: pick a non-trivial FD `X -> Y` in `R` where `X` is not a superkey. Replace `R` by:
  - `R1 = X U Y` (one copy of the offending fact)
  - `R2 = R - (Y - X)` (everything else, plus `X` to allow the join back)
- Repeat on each piece until all are in BCNF. The decomposition is **always lossless**.
- **Worked example (zip code -> city).** `R(name, street, city, zip)` with FD `zip -> city`. `zip` is not a superkey. Decompose to `R1(zip, city)` and `R2(name, street, zip)`. Both are in BCNF.

Find it in the slides: Lecture 6, slides 63-64.

1.7 Third Normal Form (3NF)

Definition. `R` is in **3NF** iff for every non-trivial FD `X -> Y` in `F+`, **at least one** of the following holds:

1. `X` is a superkey of `R`, **OR**
2. Every attribute of `Y - X` is a **prime attribute** (i.e., in some candidate key).

- 3NF is strictly weaker than BCNF: every BCNF relation is in 3NF, but not vice-versa.

- 3NF allows partial redundancy (some duplication of prime attributes) in exchange for **always being achievable while preserving all dependencies**.
- **Canonical "3NF but not BCNF" example - `dept_advisor`.** `R(s_ID, i_ID, dept_name)` with FDs `(s_ID, dept_name) -> i_ID` and `i_ID -> dept_name`. Candidate keys: `(s_ID, dept_name)` and `(s_ID, i_ID)`. Every attribute is prime. `i_ID -> dept_name` violates BCNF (`i_ID` is not a superkey) but `dept_name` is prime, so it is fine for 3NF.

Find it in the slides: Lecture 6, slides 66-67.

1.8 BCNF vs 3NF tradeoffs

Property	3NF	BCNF
Redundancy	Some allowed	Eliminated
Lossless decomposition	Yes (always achievable)	Yes (always achievable)
Dependency preservation	Yes (always achievable)	**Not always** achievable
Update anomalies	Possible	Eliminated
Strength	Weaker	Stronger

Rule of thumb during the exam.

- Check superkey-ness first. If every non-trivial FD has a superkey LHS, the relation is in BCNF (and therefore 3NF).
- If some FD `X -> Y` has a non-superkey `X`, check whether every attribute in `Y` is prime. If so, 3NF holds but BCNF does not.
- If you must decompose for BCNF and lose a dependency, decide whether the lost dependency is critical; if it is, stop at 3NF.

Find it in the slides: Lecture 6, slide 68.

1.9 1NF through 5NF in one paragraph each

- **1NF (First Normal Form).** All attribute values are **atomic** (no nested relations, no repeating groups, no comma-separated lists). This is the price of admission to the relational model.
- **2NF (Second Normal Form).** In 1NF and **no non-prime attribute is partially dependent on a candidate key** (every non-prime attribute depends on the **whole** key, not part of it). 2NF is automatic if every candidate key is a single attribute.
- **3NF.** In 2NF and no non-prime attribute is **transitively** dependent on any candidate key. Equivalent to the formal definition above.
- **BCNF.** Stronger than 3NF: the determinant of every non-trivial FD must be a superkey.
- **4NF.** In BCNF and has no non-trivial **multivalued dependency** `X ->-> Y` unless `X` is a superkey. Eliminates redundancy from independent multivalued facts (e.g., `(course, instructor)` and `(course, textbook)` independently varying).
- **5NF (Project-Join Normal Form).** Every non-trivial **join dependency** is implied by candidate keys. Eliminates redundancy from cases that decompose only into 3+ relations.

Find it in the slides: Lecture 6, slide 74.

1.10 Denormalization and materialized views

Definition. Denormalization intentionally re-introduces redundancy in a normalized schema to improve read performance.

- Trade-off: faster reads (fewer joins) at the cost of slower / more complex writes and the risk of inconsistency.
- **Materialized views** are a managed form of denormalization: the system stores the result of a query and keeps it (eventually) in sync with the base tables.
- Common in analytic and **OLAP** systems where reads dominate. Compare with **wide flat / star schema** designs (see Part 8).

Find it in the slides: Lecture 6, slides 72, 75-76.

Part 2: Query Processing (Lecture 9, slides 9-51)

2.1 Compilation pipeline

Definition. A SQL query goes through three stages: **parsing & translation -> optimization -> evaluation**.

```
SQL text -> [Parser] -> Relational Algebra Tree
        -> [Optimizer + Catalog Statistics] -> Best Physical Plan
        -> [Execution Engine] -> Result Tuples
```

- **Parser** does syntax checking and produces an internal relational-algebra-like representation.
- **Optimizer** uses **catalog statistics** (table sizes, index existence, value distributions) and a cost model to pick the cheapest plan from many equivalent ones.
- **Engine** runs the chosen plan against the storage / buffer manager.
- **`EXPLAIN <query>`** asks the optimizer to print the chosen plan without running it; **`EXPLAIN ANALYZE`** runs it and reports actual costs.

Find it in the slides: Lecture 9, slides 9-16.

2.2 Selection algorithms (A1-A10)

Definition. Algorithms for evaluating `sigma_theta(R)`. Algorithm choice depends on whether `theta` matches an index and whether the file is sorted.

These are the **textbook tags** used in Lecture 9 (slides 20-26). Match them exactly if a question asks "what is algorithm Ak?":

Tag	Algorithm	When it applies	Cost intuition
A1	Linear (file) scan	Always	`b_r` block transfers; `b_r/2` if equality on a key
A2	**Clustering (primary) index, equality on key**	Index on the search key; `attr = value` returns at most one record	`(h_i + 1) * t_B`; `~O(log N)` I/Os
A3	**Clustering index, equality on non-key**	Sorted on indexed attr; multiple matches on consecutive blocks	`h_i * t_B + t_B * b` for `b` matching blocks
A4	**Secondary index, equality (key or non-key)**	Non-clustering index; matches scattered across blocks	Single record: `(h_i + 1) * t_B`; `n` records: `(h_i + n) * t_B` (one random I/O per match)
A5	**Clustering index, comparison** (`>= v`, `<= v`)	Relation sorted on the comparison attribute	Use index to find boundary tuple, then sequential scan from there
A6	**Secondary index, comparison**	Non-clustering index, range predicate	One I/O per match; **often worse than a linear scan** if many tuples qualify
A7	**Conjunctive selection using one index**	`theta_1 AND theta_2 AND ...`; one term has a useful index	Use the most selective indexed term, fetch that candidate set, filter the rest in memory
A8	**Conjunctive selection using composite index**	A composite index covers all (or several) AND conjuncts	Single lookup against the composite key
A9	**Conjunctive selection by intersection of identifiers**	Each conjunct has an index; indexes return record pointers	Take the intersection of pointer sets, then fetch the records
A10	**Disjunctive selection by union of identifiers**	`theta_1 OR theta_2 OR ...`; **all** disjuncts have usable indexes	Take the union of pointer sets, fetch records; if **any** disjunct is unindexed, fall back to A1 (linear scan)

Important note from the deck (slide 20): binary search on a sorted file is **not** a separate algorithm here; the lecture explicitly says "binary search generally does not make sense since data is not stored consecutively" - if the file is sorted, you use A2/A5 (the clustering-index variants) instead.

Find it in the slides: Lecture 9, slides 19-26.

2.3 Conjunction vs disjunction

Conjunction `WHERE A = a AND B = b`:

- If an index exists on either `A` or `B`, use it to fetch a small candidate set and apply the remaining predicate as an in-memory filter.
- If a composite index `(A, B)` exists, use a single lookup.
- If both columns have indexes, **index intersection** can be used.

Disjunction `WHERE A = a OR B = b`:

- Indexes only help if **all** disjuncts have indexes (then take the union of the result sets, removing duplicates).
- If even one disjunct has no usable index, the optimizer must do a **full scan**, because every tuple could satisfy the un-indexed predicate.

Sample-question pattern. "Index on `name`, no index on `dept_name`." For `name = 'X' AND dept = 'CS'`, the index on `name` helps (small candidate set, filter). For `name = 'X' OR dept = 'CS'`, the index on `name` is useless because every tuple still has to be checked for `dept = 'CS'`.

Find it in the slides: Lecture 9, slide 27.

2.4 Join algorithms

Algorithm	Best when	Cost (high level)
Nested-loop	One table tiny, no useful index	`O(|R| * |S|)` tuple comparisons
Block nested-loop	Same as NL but reduces I/O by reading pages	`O(b_R * b_S)` block reads (better page buffering)
Indexed nested-loop	Inner table has an index on the join attribute	`|R| * cost(index_lookup_in_S)`; great when `R` is small and `S` is huge but indexed
Sort-merge join	Both already sorted on join attribute, or sort cost is acceptable	`O((|R| + |S|) * log(...))` if you have to sort, then linear merge
Hash join	Equi-join, neither sorted, no index, both fit working memory	Build phase: `O(|R|)`. Probe phase: `O(|S|)`. With **partitioning**, scales beyond memory; recursive partitioning if a partition still does not fit.

How hash join works.

```
Build phase:
  for each tuple r in R (the smaller / "build" relation):
    insert r into hash table H keyed by r.join_attr

Probe phase:
  for each tuple s in S (the "probe" relation):
    look up s.join_attr in H
    output every matching r joined with s
```

- **Partitioning step** is needed when `R` is too large for memory: hash both `R` and `S` on the join key into `k` partitions; for each partition `i`, run the in-memory hash join `R_i x S_i`. Each tuple is read and written at most twice.
- **Hash join is for equi-joins only.** For non-equi-joins, fall back to nested-loop or sort-merge variants.

Tiny worked example. `R(id, x) = {(1,A),(2,B),(3,C)}` (build), `S(id, y) = {(2,p),(3,q),(3,r),(4,s)}` (probe), join on `id`.

- Build: `H = {1->(1,A), 2->(2,B), 3->(3,C)}`.
- Probe: `(2,p)` finds `(2,B)` -> emit `(2,B,p)`. `(3,q)` finds `(3,C)` -> emit `(3,C,q)`. `(3,r)` finds `(3,C)` -> emit `(3,C,r)`. `(4,s)` finds nothing -> dropped.
- Output: `{(2,B,p), (3,C,q), (3,C,r)}`. Total work: build `|R|=3`, probe `|S|=4`, vs `3*4=12` for nested-loop.

Find it in the slides: Lecture 9, slides 28-41.

2.5 Outer vs inner relation in nested-loop joins

- The **outer relation** is read once. The **inner relation** is read once **per outer tuple** (or per outer block, in block-NL).
- Choose the **smaller** relation as the outer when neither is indexed; this minimizes the number of inner re-reads.
- For **indexed nested-loop**, choose the relation **without** the index as the outer and the **indexed** relation as the inner; you do `|outer|` index lookups instead of `|outer| x |inner|` comparisons.

Sample-question pattern. `R = 1,000 tuples`, `S = 1,000,000 tuples`, index on `S.join_attr`. Use **indexed nested-loop**, with `R` as the outer (1,000 lookups into `S`'s index, each O(log) or O(1)).

Find it in the slides: Lecture 9, slides 31-33.

2.6 Other operations: duplicate elimination, projection, aggregation

- **Duplicate elimination.** Done by sorting (and dropping consecutive duplicates) or hashing. SQL `DISTINCT`, `UNION` (not `UNION ALL`), and grouping all need it.
- **Projection.** Drop unused columns. The interesting cost is when the projection is followed by `DISTINCT`; the duplicate elimination dominates.
- **Aggregation.** Sort-based or hash-based. With a `GROUP BY` of moderate cardinality, hash aggregation is usually best.

Find it in the slides: Lecture 9, slides 42-44.

2.7 Materialization vs pipelining

Definition. Two strategies for executing a multi-operator plan tree.

- **Materialization.** Each operator writes its full output to a temporary table on disk; the next operator reads from that table. Pro: each operator runs to completion independently; works for any operator including blocking ones (sort, hash-build). Con: extra I/O for the temp tables; high latency before the first output tuple appears.
- **Pipelining.** Tuples flow operator-to-operator without intermediate storage. Pro: low latency; first result tuple is produced quickly; no temp tables. Con: not always possible; **blocking operators** (sort, full aggregation, hash-build phase) must materialize their input before producing any output.

Iterator (Volcano) model. Each operator implements three calls:

```
open()    -- initialize state, open inputs
next()    -- return the next output tuple, or end-of-stream
close()   -- release resources
```

Pipelining is the natural execution mode for the iterator model.

Find it in the slides: Lecture 9, slides 45-51.

Part 3: Query Optimization (Lecture 9, slides 52-77; Lecture 10, slides 7-33)

3.1 Equivalent plans and EXPLAIN

Definition. Two relational-algebra expressions are **equivalent** if they always return the same set of tuples on every database instance. The optimizer searches the space of equivalent plans for the cheapest one.

- Two plans for `SELECT name FROM instructor WHERE salary < 75000`:
  - `pi_name(sigma_{salary<75000}(instructor))` (filter, then project)
  - `sigma_{salary<75000}(pi_{name,salary}(instructor))` (project early, but must keep `salary` so the filter can run)
- The first plan is generally better if there is **no index on salary** (single scan that filters and projects), and the second is rarely a win because we still need `salary` until after the filter.

Find it in the slides: Lecture 9, slides 52-56; Lecture 10, slides 8-12.

3.2 Equivalence rules

Key rewrite rules used by the optimizer:

- Selections **commute**: `sigma_p(sigma_q(R)) = sigma_q(sigma_p(R)) = sigma_{p AND q}(R)`.
- Selection distributes over **natural join** if the predicate references only one side: `sigma_p(R |x| S) = sigma_p(R) |x| S`. This is **selection pushdown** - the most important heuristic.
- **Projection** can be pushed too, but you must keep all attributes needed by later operators.
- Inner joins are **commutative** and **associative**: `R |x| S = S |x| R` and `(R |x| S) |x| T = R |x| (S |x| T)`.
- **Outer joins are NOT freely commutative or associative.** Pushing predicates through outer joins can change the result.

Pushdown example. `sigma_{salary > 100K}(instructor |x| department)` rewrites to `sigma_{salary > 100K}(instructor) |x| department`, because `salary` lives only in `instructor`. If 5% of instructors qualify, the join sees ~5% of the rows it would otherwise. The output is identical; the cost can be 20x lower.

Find it in the slides: Lecture 9, slides 57-73; Lecture 10, slides 13-25.

3.3 Heuristic: push selections, project early

The optimizer applies cheap heuristics first:

1. Push selections as deep as possible (toward base tables).
2. Project away unused columns as early as possible.
3. Combine selections and Cartesian products into joins (avoid materializing huge cross products).
4. Replace selection over union/intersection with selection over each operand.

These heuristics shrink intermediate results, which is the dominant cost factor.

Find it in the slides: Lecture 10, slides 17-22.

3.4 Join order enumeration: dynamic programming

Definition. With `n` relations, there are roughly `n!` join orders; the optimizer must search this space cheaply.

- The standard approach is **bottom-up dynamic programming** (Selinger / System R):
  - Compute the best plan for every **single relation** (e.g., index scan vs full scan).
  - For every pair, every triple, …, every `k`-relation subset, compute the best plan by combining smaller best plans.
  - Memoize by subset of relations to avoid recomputation.
- Pseudocode `findbestplan(S)`:

```
if bestplan[S] is already computed, return it
if |S| == 1, compute best access path for S and store in bestplan
else:
  for each non-empty proper subset S1 of S:
    P1 = findbestplan(S1)
    P2 = findbestplan(S - S1)
    consider plan P1 join P2 (and the reverse) using each join algorithm
  store the cheapest in bestplan[S]
return bestplan[S]
```

- For very large `n`, optimizers use **left-deep tree restrictions**, **greedy join ordering**, or **genetic algorithms**.

Find it in the slides: Lecture 9, slides 74-77; Lecture 10, slides 26-30.

3.5 Cost estimation: selectivity and statistics

Definition. Selectivity of a predicate `p` is the fraction of tuples that satisfy it.

- The optimizer maintains **catalog statistics**: number of tuples, number of distinct values per column, histograms of value distributions, average tuple width.
- Cost of a node = `I/O cost (block reads/writes) + CPU cost (tuple comparisons)`. I/O usually dominates.
- Common assumptions when statistics are missing: **uniform distribution** of values, **independence** of predicates. These are often wrong, which is why optimizers occasionally pick poor plans.
- **Why a per-operator-optimal plan may not be globally optimal:** picking the cheapest plan for each subexpression in isolation can lead to a plan whose intermediate results are produced in a poor order or sort, so the overall plan has more work than a slightly suboptimal sub-plan that produces sorted output for a downstream merge join.

Find it in the slides: Lecture 9, slide 76; Lecture 10, slides 31-33.

3.6 Why an index is worth its cost

- Indexes accelerate point lookups and range scans (`O(log N)` for B+-tree, near `O(1)` for hash).

- Cost: indexes consume disk space and slow down inserts/updates/deletes (every modification updates each affected index).
- An index is worthwhile when **read-to-write ratio is high** and the column has good selectivity. For low-selectivity predicates (e.g., a boolean column with 50/50 split), a scan beats an index.

Find it in the slides: Lecture 9, slide 19; review of B+-tree from earlier modules.

Part 4: Transactions and Recovery (Lecture 10, slides 34-56)

4.1 ACID properties

Letter	Property	One-line meaning
A	Atomicity	All actions of the transaction commit, or none do.
C	Consistency	The transaction takes the DB from one valid state to another (constraints, FKs, app invariants).
I	Isolation	Concurrent transactions appear to run serially (no interference).
D	Durability	Once committed, the effects survive crashes, power loss, OS reboots.

The textbook A-to-B transfer example. Move $50 from `A` to `B`:

```
BEGIN
  read(A); A := A - 50; write(A)
  read(B); B := B + 50; write(B)
COMMIT
```

If the system crashes after the `write(A)` but before the `write(B)`, the $50 has vanished and the bank is short. **Atomicity** is violated. The DBMS prevents this through **logging + recovery** (see 4.4-4.5).

Find it in the slides: Lecture 10, slides 34-43.

4.2 Transaction states

```
ACTIVE -> PARTIALLY COMMITTED -> COMMITTED
   |              |
   v              v
FAILED -------> ABORTED
```

- **Active.** Executing.
- **Partially committed.** Last action done, but commit record not yet on stable storage.

- **Committed.** Commit record forced to log; effects are durable.
- **Failed.** Hit an error; cannot proceed.
- **Aborted.** Rollback complete; either restart or abandon.

Find it in the slides: Lecture 10, slides 41-43.

4.3 Why a naive "commit then write to disk" approach fails

Direct write-on-commit is unworkable because:

- Disk I/O is expensive; flushing each transaction's pages on commit kills throughput.
- The buffer manager already steals dirty pages back to disk **before** commit (the **steal** policy), and may not have flushed them by commit time (**no-force**).
- Without a log, after a crash the DBMS cannot tell which writes were from committed vs uncommitted transactions.

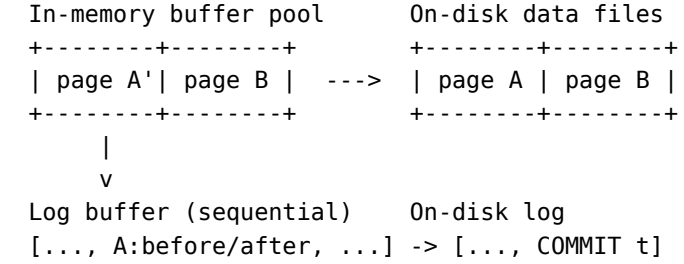
The fix: **write-ahead logging (WAL)**.

Find it in the slides: Lecture 10, slides 44-49.

4.4 Write-Ahead Logging (WAL)

The WAL rule. Before any data page is written to disk, the corresponding **log records** must already be on stable storage.

- The **log** is a sequential file of `<txn_id, page_id, offset, before_image, after_image>` records (each tagged with a unique **LSN**, log sequence number).
- For each update: write the log record to the in-memory log buffer **and** modify the buffer page; tag the page with the LSN of its latest log record.
- **The WAL rule applies at flush time:** before flushing a dirty data page to disk, all log records up to that page's LSN must already be on disk. (The in-memory order of "modify page" vs "append log record" is not what matters; the disk order is.)
- For commit: write a `<COMMIT t>` record and **force the log** to disk. Once that fsync returns, the transaction is durable.
- **Steal / no-force buffer policy:** dirty pages from uncommitted transactions can be written to disk early ("steal"); pages from committed transactions are not forced to disk on commit ("no-force"). This decouples buffer management from durability.



Find it in the slides: Lecture 10, slides 50-53.

4.5 ARIES: analysis, redo, undo

After a crash, the recovery manager runs **three passes** over the log:

1. **Analysis pass** (forward from the last checkpoint). Determines which transactions were active at the crash, which dirty pages were in the buffer, and the right starting point for redo.
2. **Redo pass** (forward). Replays **every** logged action whose effect may not have reached disk yet, including those of transactions that ultimately aborted - this restores the DB to its exact state at the moment of the crash.
3. **Undo pass** (backward, in reverse log order). For every transaction that was still active at the crash, undo its actions (using the before-images in the log), writing **compensation log records (CLRs)** so undo itself is idempotent if a second crash occurs mid-recovery.

Net effect: committed transactions are preserved (Durability, Atomicity); uncommitted transactions are completely rolled back (Atomicity).

Walked-through log example. Suppose the on-disk log at the moment of the crash is:

```
LSN 10  T1 update p1: A -> A'
LSN 20  T2 update p2: B -> B'
LSN 30  T1 commit
LSN 40  T2 update p3: C -> C'
                                <CRASH>
```

- **Analysis** sees `T1` has a commit record, `T2` does not. Active set = `{T2}`.
- **Redo** replays LSN 10, 20, 40 *unconditionally* - the buffer pages may or may not have reached disk, but reapplying logged updates is safe.
- **Undo** rolls `T2` back, in reverse log order: undo LSN 40 (write CLR), undo LSN 20 (write CLR). `T1`'s update at LSN 10 stays because `T1` committed.

Find it in the slides: Lecture 10, slide 54.

4.6 Durability mechanisms beyond the log

- **RAID** (Redundant Array of Independent Disks) - mirroring or parity-protected disks survive a single drive failure.
- **Duplex writes** - the same write is sent to two independent disks; the write is acknowledged only when both succeed.
- **Replication** to remote nodes:
  - **Active / passive (primary-backup).** All writes go to the primary; a secondary applies the log shipped from the primary. Failover promotes the secondary.
  - **Active / active (multi-master).** Multiple nodes accept writes; conflicts must be resolved (timestamps, vector clocks, application logic). Higher availability but harder consistency.
- These choices feed directly into **CAP** (see 6.2).

Find it in the slides: Lecture 10, slides 55-56.

Part 5: Concurrency and Isolation (Lecture 10 slides 57-69; Lecture 11 slides 8-43; Lecture 12-V2 slides 6-20)

5.1 Schedules: serial vs concurrent

Definition. A **schedule** is the interleaved sequence of operations from a set of transactions. A **serial schedule** runs transactions one at a time, end-to-end; concurrent schedules interleave their operations.

- Serial schedules are trivially correct (they obey ACID by construction) but waste resources and slow down the system.
- Concurrent schedules can produce **anomalies** (see 5.5) unless concurrency control enforces some equivalent of serial execution.

Find it in the slides: Lecture 10, slides 57-60; Lecture 11, slides 12-23.

5.2 Conflict serializability and the precedence graph

Definition. Two operations **conflict** if (a) they are from different transactions, (b) they access the same data item, and (c) at least one is a write.

- A schedule is **conflict serializable** iff it can be transformed into a serial schedule by swapping non-conflicting adjacent operations.
- **Precedence (serialization) graph.** Nodes = transactions. Edge `Ti -> Tj` whenever `Ti` has a conflicting operation that precedes a conflicting operation of `Tj` on the same item.
- **Theorem.** A schedule is conflict serializable **iff its precedence graph is acyclic**. A topological sort of an acyclic graph gives an equivalent serial order.

Worked cycle example. Consider the schedule

```
Time -->  1       2       3       4
T1:       R(A)            W(B)
T2:               W(A)            R(B)
```

The conflicts are:

- `T1.R(A)` precedes `T2.W(A)` -> edge `T1 -> T2`
- `T1.W(B)` precedes `T2.R(B)` -> edge `T1 -> T2` (no cycle yet)

Now flip the second pair:

```
Time -->  1       2       3       4
T1:       R(A)                    W(B)
T2:               W(A)    R(B)
```

- `T1.R(A)` precedes `T2.W(A)` -> edge `T1 -> T2`
- `T2.R(B)` precedes `T1.W(B)` -> edge `T2 -> T1`

The precedence graph now has the cycle `T1 -> T2 -> T1`, so this schedule is **NOT conflict serializable**.

```
T1 ------> T2
 \________/
   (cycle)
```

Find it in the slides: Lecture 11, slides 18-33.

5.3 Conflict vs view serializability

- **View serializability** is a strictly broader correctness criterion. A schedule is view-equivalent to a serial schedule if (a) the same initial reads, (b) the same read-from relationships for every write, (c) the same final writes.
- Every conflict-serializable schedule is view-serializable; the converse is not true (some view-serializable schedules with **blind writes** have a cycle in the precedence graph).
- **Why DBMSs use conflict serializability.** Testing view serializability is NP-complete; testing conflict serializability is `O(V + E)` (cycle detection). Conflict serializability is the practical correctness criterion.

Find it in the slides: Lecture 11, slide 24.

5.4 Recoverable, cascadeless, and strict schedules

Property	Rule	What it prevents
Recoverable	If `Tj` reads a value written by `Ti`, then `Tj` commits **after** `Ti` commits.	The need to roll back an already-committed transaction (which would violate Atomicity / Durability).
Cascadeless (ACA)	A transaction may only read values written by **committed** transactions.	Cascading rollbacks (one abort triggers many).
Strict	A value written by `Ti` cannot be read **or overwritten** by any other transaction until `Ti` commits or aborts.	Anything that complicates undo by before-image.

- **Cascading rollback.** `T1` writes `X`; `T2` reads `X`; `T3` reads what `T2` derived; …; `T1` aborts -> all of `T2, T3, ...` must also abort. Catastrophic for throughput.
- **Cascadeless schedules** prevent this by holding writes "private" until commit.
- **Strict schedules** allow undo to be done with before-images alone (no need to undo intermediate writes).

Three-line examples.

Schedule	Property
`T1: W(A); T2: R(A); T2: COMMIT; T1: ABORT`	**Not recoverable** - `T2` committed reading uncommitted data, but now `T1` aborts. We would have to "uncommit" `T2`.
`T1: W(A); T2: R(A); T1: COMMIT; T2: COMMIT`	**Recoverable but not cascadeless** - `T2` read uncommitted data, but luckily `T1` committed first. If `T1` had aborted, `T2` would have to abort too (cascade).
`T1: W(A); T1: COMMIT; T2: R(A); T2: COMMIT`	**Cascadeless** (and strict, since no overwrite/read of `A` happened before `T1` committed).

Find it in the slides: Lecture 11, slides 34-36.

5.5 Strict Two-Phase Locking (Strict 2PL)

Definition. A locking protocol where (1) a transaction holds a **shared (S)** lock to read and an **exclusive (X)** lock to write; (2) once it releases any lock it cannot acquire any new locks (the "two phases": growing then shrinking); (3) **all locks are held until commit or abort** (the "strict" part).

- Under Strict 2PL, every legal schedule is **conflict serializable, recoverable, cascadeless, and strict**.
- Why it serializes: the `lock_held_until_commit` rule means that any conflicting operation in another transaction must wait for the first to commit, so the serialization order is exactly the commit order.
- Why it simplifies recovery: because schedules are strict, `undo` only needs the before-image; no chains of dependent writes to chase.
- The price: **deadlocks** can occur (see 5.8).

Lock compatibility matrix.

Held \ Requested	S (shared)	X (exclusive)
none	grant	grant
S	grant	wait
X	wait	wait

Schedule under Strict 2PL. `T1` and `T2` both want to update `A` then `B` (the classic transfer scenario):

```
T1: lock-X(A); R(A); W(A); lock-X(B); R(B); W(B); commit; unlock A,B
T2:                                  lock-X(A) -> WAITS for T1
                                                  T1 commits, T2 acquires X(A)
                                  R(A); W(A); lock-X(B); R(B); W(B); commit;
unlock
```

`T2` waits at its first lock request because `T1` still holds `X(A)`. The protocol forces the schedule into the serial order `T1 -> T2`. If both transactions had instead grabbed locks in different orders (e.g., `T1` locks `A` then `B`, `T2` locks `B` then `A`), they would **deadlock** - see 5.8.

Find it in the slides: Lecture 11, slides 37-38; Lecture 12-V2, slides 8-12.

5.6 SQL isolation levels and the anomalies they (do not) prevent

Level	Dirty read	Non-repeatable read	Phantom read	Lost update / write skew
READ UNCOMMITTED	possible	possible	possible	possible
READ COMMITTED	prevented	possible	possible	possible
REPEATABLE READ	prevented	prevented	possible (range locks not managed in the lecture's description; in real MySQL InnoDB next-key/gap locks usually prevent this)	write skew possible under snapshot isolation
SERIALIZABLE	prevented	prevented	prevented	prevented

- **Dirty read.** Read of a value written by a yet-uncommitted transaction. *Example:* `T1: balance := balance + 1000` (uncommitted); `T2: SELECT balance` (sees +1000); `T1 ROLLBACK`. `T2` made decisions on a phantom value.
- **Non-repeatable read.** Reading the same row twice in one transaction returns different values (because another committed in between). *Example:* `T1: SELECT price FROM item WHERE id=1` -> 10; `T2: UPDATE item SET price=12 WHERE id=1; COMMIT`; `T1: SELECT price FROM item WHERE id=1` -> 12. Same query, two answers.
- **Phantom read.** Re-running the same range query returns a different set of rows (because another transaction inserted/deleted matching rows). *Example:* `T1: SELECT * FROM emp WHERE dept='CS'` -> 3 rows; `T2: INSERT INTO emp(...,'CS'); COMMIT`; `T1: SELECT * FROM emp WHERE dept='CS'` -> 4 rows. The new row is a "phantom."
- **Write skew.** Two transactions read overlapping data and write disjoint rows under snapshot isolation, producing an outcome no serial schedule could produce. *Example (on-call doctors):* invariant "at least one doctor on call." Both `T1` and `T2` see two doctors on call, each decides "I can take myself off"; both write disjoint rows; the invariant is now violated, but each saw a "consistent" snapshot.

Snapshot isolation caveat (Oracle, older PostgreSQL). Snapshot isolation prevents dirty / non-repeatable / phantom reads but allows **write skew** - and was historically advertised as "Serializable" by some vendors even though it is not.

Find it in the slides: Lecture 10, slides 67-68; Lecture 11, slides 39-43.

5.7 Cursors vs REST: stateless APIs and isolation

- **Cursors** are server-side, stateful pointers to a result set. Inside a single transaction with `REPEATABLE READ` or stronger, they give consistent reads.
- **REST** is **stateless** by design: every HTTP request is independent. There is no per-client cursor maintained by the server.
- Implication for paging: a paged REST API that "remembers where you were" must encode the position into the request itself (e.g., `?after=last_id`), and is **not** isolation-protected against concurrent inserts/deletes.

Find it in the slides: Lecture 10, slide 69; Lecture 11, slide 44.

5.8 Deadlock: prevention, avoidance, detection, recovery

Definition. Deadlock occurs when a set of transactions are each waiting for a lock held by another, forming a cycle. None can proceed.

```
T1 holds X-lock on A, requests X-lock on B
T2 holds X-lock on B, requests X-lock on A
       ----> circular wait ----
```

Wait-for graph. Nodes = transactions. Edge `Ti -> Tj` whenever `Ti` is waiting for a lock held by `Tj`. **A cycle means a deadlock.** The recovery manager runs cycle detection periodically and aborts a victim to break the cycle.

Prevention techniques (no deadlocks ever, by construction):

- **Resource ordering.** Define a total order on data items; require every transaction to acquire locks in that order. No cycle can ever form.
- **Timestamp-ordered protocols using transaction timestamps `TS(T)`** (older = smaller TS):
  - **Wait-die (non-preemptive).** If `T_i` requests a lock held by `T_j`: if `TS(T_i) < TS(T_j)` (T_i is older), `T_i` waits; otherwise, `T_i` is **aborted ("dies")** and restarts with the same TS.
  - **Wound-wait (preemptive).** If `TS(T_i) < TS(T_j)`, `T_i` **wounds** `T_j` (forces `T_j` to abort); otherwise, `T_i` waits.
  - In both, **older transactions are favored**, so no transaction can be repeatedly aborted forever (no starvation): on restart it keeps its old (low) timestamp.

  Worked example. `TS(T1)=10` (older), `TS(T2)=20` (younger). `T1` holds `A`; `T2` holds `B`. Then both ask for the other's lock:
  - **Wait-die:** `T1` requests `B` (held by younger `T2`) -> older requesting younger, so `T1` **waits**. `T2` requests `A` (held by older `T1`) -> younger requesting older, so `T2` **dies** (aborts, restarts with `TS=20`).

  - **Wound-wait:** `T1` requests `B` (held by younger `T2`) -> older preempts younger, so `T1` **wounds** `T2` (kills it). `T2` restarts. If instead `T2` had asked first: younger requesting older -> `T2` **waits**.
- **Timeouts.** Simplest: any transaction that waits too long is aborted. Cheap, but tunes badly (false positives on heavy load, real deadlocks linger if timeout too long).

Detection + recovery (allow deadlocks, fix after the fact):

- Build the wait-for graph; run cycle detection.
- On a cycle, choose a **victim** to abort. Heuristics: youngest transaction, transaction with fewest writes, transaction with shortest progress.
- **Total rollback** vs **partial rollback** to a savepoint.
- Guard against **starvation** by tracking how many times a transaction has been victimized and avoiding chronic victims.

Find it in the slides: Lecture 12-V2, slides 6-20.

Part 6: Distributed Consistency and Scaling (Lecture 10 slides 70-79; Lecture 11 slides 44-54)

6.1 Eventual consistency and BASE

Definition. Eventual consistency is a weaker liveness guarantee: in the absence of new writes, all replicas will **eventually** converge to the same value, but reads in the meantime may be stale.

- **BASE** is the design philosophy that pairs with eventual consistency:
  - **B**asically **A**vailable - the system always responds, even if with stale data.
  - **S**oft state - state may change without input as replicas reconcile.
  - **E**ventually consistent - convergence is the goal, not strict immediate consistency.

Trait	ACID	BASE
Consistency	Strong, immediate	Eventual
Availability	Lower (locks, coordination)	High
Latency	Higher (synchronous coordination)	Lower
Partition tolerance	Limited (coordination needed)	High
Typical workload	OLTP, financial	Web-scale, social, IoT, analytics

When to prefer BASE. When availability and partition tolerance trump immediate consistency: e.g., a globally distributed product catalog, a like counter, an analytics ingestion pipeline.

Concrete example. A user adds an item to a shopping cart. The write hits the `us-east` replica and returns success. A second later, the same user reads from the `eu-west` replica and may *not yet* see the item (replication lag). After replication completes, all replicas converge. This is acceptable

for a cart, a “like” count, or a follower count - and unacceptable for a bank balance, where strong consistency is mandatory.

Find it in the slides: Lecture 10, slides 70-74; Lecture 11, slides 45-50.

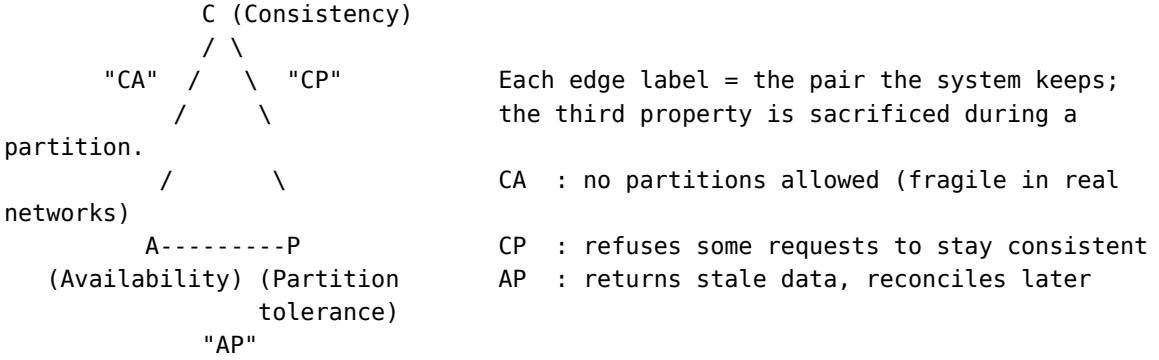
6.2 The CAP theorem

Definition (lecture wording, Lecture 10 slide 75 / Lecture 11 slide 50). In any **distributed** data system, you can guarantee at most **two** of:

- **C**onsistency - every read receives the most recent write or an error.
- **A**vailability - every request receives a (non-error) response, without a guarantee that it contains the most recent write.
- **P**artition tolerance - the system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.

Because partitions in a real network are unavoidable, the practical choice is **CP vs AP**:

- **CP** systems sacrifice availability during a partition (refuse some requests to stay consistent). Examples: traditional RDBMS clusters with synchronous replication, MongoDB with `majority` writes, HBase.
- **AP** systems sacrifice consistency during a partition (return possibly stale data, reconcile later). Examples: DynamoDB, Cassandra (tunable), CouchDB.



Because partitions in a real network are unavoidable, **CA is rarely a real choice**; pick **CP** or **AP**.

Concrete example. A bank has a NY primary and an SF replica. The transatlantic link drops.

- **CP choice:** SF refuses writes (and possibly reads) until the link is restored. Customers in California get error pages; the two sides cannot diverge.
- **AP choice:** Both NY and SF accept writes locally. Both stay available, but a customer’s balance now exists in two diverged versions; reconciliation logic (last-write-wins, version vectors, conflict-resolution policies) merges them when the partition heals.

Find it in the slides: Lecture 10, slide 75; Lecture 11, slides 51.

6.3 Scale-up vs scale-out

	Scale-up (vertical)	Scale-out (horizontal)
Mechanism	Bigger machine: more CPU, RAM, faster disks	More machines, sharding / partitioning
Limit	Hardware ceiling, expensive per unit	Coordination cost, network latency
Failure	Single point of failure	Built-in redundancy if replicated
Code changes	None (transparent)	Often significant (consistency, joins, transactions)
Cost curve	Super-linear (high-end hardware is expensive)	Roughly linear

Why scale-out is hard for relational operations.

- Joins between tables on different nodes require shipping data across the network (a "shuffle"), which is far slower than local I/O.
- Distributed transactions need **two-phase commit (2PC)**, which is fragile in the presence of partitions.
- Indexes that span shards become global structures with their own distributed-system problems.

Find it in the slides: Lecture 10, slides 76-79; Lecture 11, slides 52-54.

6.4 Shared-nothing vs shared-disk; sharding and partitioning

- **Shared-disk.** Multiple compute nodes attached to the same shared storage (e.g., Oracle RAC). Easier consistency, harder scaling beyond the storage layer.
- **Shared-nothing.** Each node owns its own CPU, memory, and disks; coordination is by message passing. Used by virtually every modern OLAP / NoSQL system.
- **Sharding (MongoDB)** - documents are partitioned across shards by a **shard key** (range or hash). Each shard is itself a replica set.
- **Partitioning (DynamoDB)** - items are partitioned by **partition key** (hashed to a partition). Each partition is replicated three ways across availability zones.
- **Trade-off.** Sharding scales reads/writes nearly linearly with the number of shards as long as the workload is well-distributed by the key. Hot keys, cross-shard transactions, and cross-shard joins are the failure modes.

Concrete example. An `orders` collection sharded by hashed `customer_id` across three shards `A, B, C`:

- All orders for `customer_id = 42` live on whichever shard `hash(42) mod 3` selects (say `B`); each shard is itself a 3-node replica set.
- A query `db.orders.find({customer_id: 42})` is routed by `mongos` to **only shard B**.

- A query `db.orders.find({status: "PAID"})` has to **fan out** to all three shards (no shard-key filter), and `mongos` merges the results.
- A bad shard-key choice - e.g., `created_date` - creates a hot partition: every new order goes to today's shard. Hashed keys avoid this.

Find it in the slides: Lecture 10, slides 76-79.

Part 7: NoSQL (Lecture 9, slides 78-101)

7.1 The four common NoSQL categories

Category	Model	Examples	Typical use
Document	JSON-like docs in collections; flexible schema	MongoDB, Couchbase	Content, catalogs, user profiles
Key-value	Pure `key -> opaque value`	Redis, DynamoDB, RocksDB	Caches, sessions, simple lookups
Wide-column	Sparse table; rows have arbitrary columns under column families	Cassandra, HBase, BigTable	Time series, write-heavy logs
Graph	Nodes + edges + properties; query along paths	Neo4j, Neptune	Social, recommendations, fraud

The categories overlap (DynamoDB is "key-value" with optional document attributes; MongoDB has key-value-style usage). Pick by **query pattern**, not marketing.

Find it in the slides: Lecture 9, slides 79-80.

7.2 MongoDB data model

Hierarchy. `mongod` instance -> **database** -> **collection** -> **document** -> **field** (-> nested document or array).

- A **document** is a BSON object (binary JSON). Documents in the same collection do **not** need identical fields.
- Every document has a unique `_id`. By default, MongoDB assigns an `ObjectId`.
- Tools: `mongod` (server), `mongosh` (shell), Compass (GUI), `pymongo` (Python driver).
- **SQL ↔ Mongo loose mapping:**

SQL	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field
Primary key	`_id`
Join	`$lookup` (aggregation)
Foreign key	Reference (no enforced FK)

Find it in the slides: Lecture 9, slides 81-87.

7.3 CRUD with `find`, projection, updates, deletes

Read. `db.collection.find(filter, projection)`.

```
// Filter: students whose dept is CS and score > 80
// Projection: name only, suppress _id
db.students.find(
  { dept: "CS", score: { $gt: 80 } },
  { name: 1, _id: 0 }
)
```

Insert. `db.col.insertOne({...})`, `db.col.insertMany([{...}, ...])`.

Update. `db.col.updateOne(filter, { $set: { ... } })`, `updateMany`, `replaceOne`.

Delete. `db.col.deleteOne(filter)`, `deleteMany(filter)`.

- The **filter** is a document where each field is matched by equality or by a query operator (`$gt`, `$lt`, `$in`, `$regex`, `$elemMatch`, ...).
- The **projection** uses `1` to include and `0` to exclude (you cannot mix include and exclude except with `_id`).

Find it in the slides: Lecture 9, slides 83-88.

7.4 Aggregation pipeline

Definition. A pipeline of stages, each consuming the previous stage's output. Common stages:

Stage	Effect
`$match`	Filter documents (like `WHERE`)
`$project`	Reshape documents (like `SELECT`)
`$group`	Group by an expression and aggregate (like `GROUP BY`)
`$sort`	Sort
`$limit` / `$skip`	Pagination
`$lookup`	Left outer join against another collection
`$unwind`	Explode an array field into one document per element

```
db.orders.aggregate([
  { $match: { status: "PAID" } },
  { $group: { _id: "$customerId", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } },
  { $limit: 10 }
])
```

Find it in the slides: Lecture 9, slides 89-91.

7.5 Indexes, replication, sharding (MongoDB)

- **Indexes** - B-tree by default; built on any field, including nested fields and array elements (multikey index). Without an appropriate index, queries do a **collection scan**.
- **Replication** - a **replica set** is a primary plus secondaries. Writes go to the primary, then replicate to secondaries asynchronously. Failover elects a new primary if the current one fails.
- **Sharding** - the **shard key** determines which shard each document lives on. Range-based or hashed. A `mongos` query router fans queries out to relevant shards.
- **CAP positioning** *(real-world note, not from the deck).* With `writeConcern: majority` and `readConcern: majority`, MongoDB acts as a **CP** system; with relaxed concerns, more **AP**.

Find it in the slides: Lecture 9, slide 90.

7.6 Neo4j and Cypher patterns

Definition. A graph database stores **nodes** (entities), **relationships** (typed, directed edges with properties), and **labels**. Queries are written in **Cypher**.

- `(:Label {prop: value})` matches a node with a label and property; `-[:REL_TYPE]->` matches a typed directed edge.
- Patterns express graph traversals declaratively; the engine handles search.

Common exam-relevant patterns.

- **Who acted in which movies.**

```
MATCH (a:Person)-[:ACTED_IN]->(m:Movie)
RETURN a.name, m.title
```

- **Co-stars of an actor (recommendations / second-degree).** The `WHERE co <> me` clause excludes the actor themselves from the result.

```
MATCH (me:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m:Movie)
      <-[:ACTED_IN]-(co:Person)
WHERE co <> me
RETURN co.name, count(m) AS shared
ORDER BY shared DESC
```

- **Path between Tom Hanks and Tom Cruise (shortest co-actor chain).**

```
MATCH p = shortestPath(
  (a:Person {name:"Tom Hanks"})-[:ACTED_IN*]-(b:Person {name:"Tom Cruise"})
)
RETURN p
```

- **Six degrees of Kevin Bacon (Bacon number = path length / 2 along ACTED_IN).**

```
MATCH p = shortestPath(
  (a:Person {name:"Kevin Bacon"})-[:ACTED_IN*]-(x:Person {name:"<someone>"})
)
RETURN length(p)/2 AS bacon_number
```

Find it in the slides: Lecture 9, slides 92-101.

Part 8: Big Data and Analytics (Lecture 11 slides 55-77; Lecture 12-V2 slides 21-56)

8.1 The 5 V's of big data

V	Meaning
Volume	Sizes that exceed a single machine's capacity
Velocity	Rate of arrival; streaming vs batch
Variety	Structured + semi-structured + unstructured
Veracity	Trustworthiness, noise, bias
Value	The business/scientific output the data enables

Find it in the slides: Lecture 11, slide 58; Lecture 12-V2, slide 24.

8.2 Enterprise information integration (EII)

Definition. EII is the discipline of unifying data from many operational systems (CRM, ERP, web, sensors, SaaS) into a single analytic surface.

- The hard part is rarely "moving bits"; it is **aligning schemas, semantics, units, granularity, and time**.
- EII produces a **single source of truth** for analytics that never modifies the operational systems.

Find it in the slides: Lecture 11, slide 57; Lecture 12-V2, slides 22-23.

8.3 Data warehouse vs data lake

	Warehouse	Lake
Schema	Schema-on-write (defined up front)	Schema-on-read (discovered later)
Data	Cleaned, conformed, structured	Raw, mixed formats, semi-/unstructured
Workload	OLAP / BI / dashboards	ML, exploratory analytics, reprocessing
Storage	Columnar warehouse (Snowflake, Redshift, BigQuery)	Object store (S3, GCS, ADLS)
Cost	Higher per-TB	Lower per-TB
Lakehouse trend	Combine the two: structured layer (Delta, Iceberg, Hudi) on top of object store	

Find it in the slides: Lecture 11, slides 59-61; Lecture 12-V2, slides 25-27.

8.4 ETL vs ELT

ETL = Extract, Transform, Load. Source -> staging area where transformations happen -> target warehouse. Used historically when warehouse compute was scarce/expensive.

ELT = Extract, Load, Transform. Source -> raw landing zone in the warehouse/lake -> transformations run **inside** the warehouse using SQL or notebook engines (dbt, Spark). Used now because warehouse compute is cheap and elastic.

	ETL	ELT
Where transform runs	External engine	Inside warehouse
Storage of raw data	Often discarded	Always retained
Schema timing	Schema-on-write	Schema-on-read possible
Reprocessing	Hard (raw lost)	Easy (raw kept)

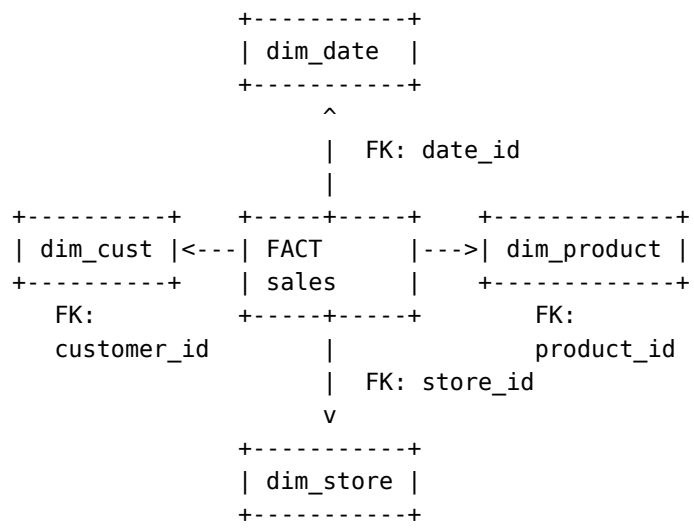
Find it in the slides: Lecture 11, slides 62-63; Lecture 12-V2, slide 27.

8.5 Star schema: fact and dimension tables

Definition. The standard analytic schema.

- **Fact table.** A row per **business event** (sale, click, payment). Contains:
  - Foreign keys to dimension tables.
  - **Measures** - additive numeric values (`amount`, `quantity`, `duration`).

- **Dimension tables.** Wide, descriptive, slowly changing tables that describe the **context** of facts (`customer`, `product`, `date`, `store`).



(Arrows point from the fact table to each dimension, the direction of the foreign-key reference.)

Example #1 - the lecture's Classicmodels star schema (Lecture 12-V2 slide 71, HW4). This is the schema you should reproduce if a sample question references the Classicmodels sales warehouse:

- **Facts (measures):** `quantityOrdered`, `priceEach`, and the derived measure `revenue = quantityOrdered * priceEach`. The fact table joins these measures to its dimension FKs.
- **Three dimensions:**
  - `Location(region, country, city)` - location of the customer placing the order.
  - `Date(year, quarter, month)` - hierarchical date dimension.
  - `Product(productLine, productScale)` - product taxonomy.
- **Source operational tables that feed the fact table:** `Customers`, `Orders`, `Orderdetails`, `Products`.

Example #2 - a generic sales warehouse (textbook style). Useful if the question is open-ended:

- One **fact**: `fact_sales(sale_id, date_id, customer_id, product_id, store_id, quantity, amount)`. Surrogate `sale_id` plus FK columns and additive measures.
- Four typical **dimensions**:
  - `dim_customer(customer_id, name, segment, country)`
  - `dim_product(product_id, name, category, brand)`
  - `dim_date(date_id, day, month, quarter, year)`
  - `dim_store(store_id, store_name, city, region, country)`

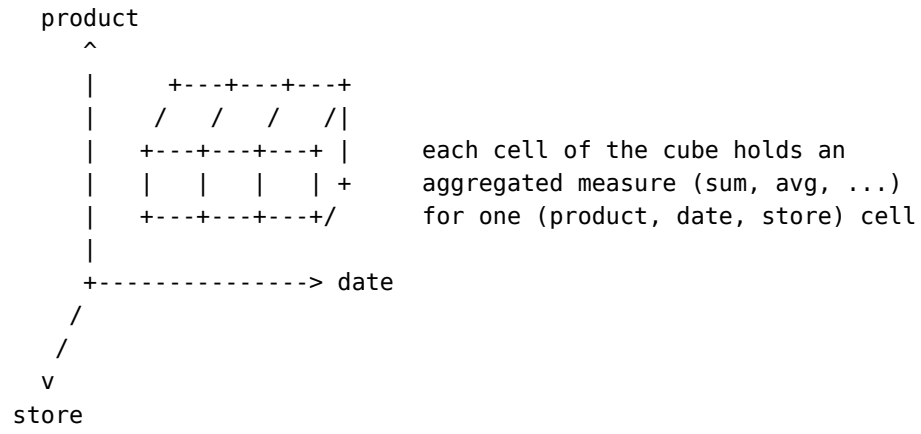
Why useful for analytics. Most analytic queries reduce to "select measures from fact, group by dimension attributes" - a tractable shape that columnar engines optimize aggressively. The schema is **wide and denormalized on purpose** (denormalization for read speed; see 1.10).

Snowflake schema - dimensions are themselves normalized into sub-dimensions. Saves space; costs join effort. Star is preferred unless dimension cardinality is enormous.

Find it in the slides: Lecture 12-V2, slides 29-31 (concept) and slide 71 (the Classicmodels facts/dimensions for HW4).

8.6 OLAP and the data cube

Definition. OLAP (Online Analytical Processing) views the fact table as a multi-dimensional **data cube**. Each axis is a dimension; each cell is the aggregated measure for that combination of dimension values.



Five OLAP operators.

Operator	What it does	SQL analog
Slice	Fix one dimension to a single value	`WHERE date = '2026-Q1'`
Dice	Restrict multiple dimensions to subsets	`WHERE region IN ('US','EU') AND product IN (...)`
Roll-up	Aggregate up a hierarchy (city -> region -> country)	`GROUP BY country`
Drill-down	The opposite of roll-up; go to finer granularity	`GROUP BY city`
Pivot	Rotate dimensions to put a different one on rows/cols	Crosstab; `PIVOT` operator

Hierarchies in dimensions enable roll-up and drill-down (e.g., `date: day -> month -> quarter -> year`; `store: store -> city -> region -> country`).

Concrete examples on the Classicmodels star schema (`fact_sales` joined to `Date`, `Location`, `Product` dimensions; measure = `revenue`):

```sql
-- Slice (fix one dimension to a single value)
SELECT productLine, SUM(revenue)
FROM fact_sales JOIN Date USING(date_id) JOIN Product USING(product_id)
WHERE year = 2024
GROUP BY productLine;

-- Dice (filter on multiple dimensions to subsets)
SELECT region, productLine, SUM(revenue)
FROM fact_sales JOIN Location USING(location_id) JOIN Product USING(product_id)
WHERE region IN ('EMEA','NA') AND productLine IN ('Classic Cars','Trucks')
GROUP BY region, productLine;

-- Roll-up (aggregate up the location hierarchy: city -> country)
SELECT country, SUM(revenue)
FROM fact_sales JOIN Location USING(location_id)
GROUP BY country;

-- Drill-down (finer granularity within France)
SELECT city, SUM(revenue)
FROM fact_sales JOIN Location USING(location_id)
WHERE country = 'France'
GROUP BY city;

-- Pivot (year on rows, productLine on columns)
SELECT year,
  SUM(CASE WHEN productLine='Classic Cars' THEN revenue END) AS classic_cars,
  SUM(CASE WHEN productLine='Trucks'       THEN revenue END) AS trucks
FROM fact_sales JOIN Date USING(date_id) JOIN Product USING(product_id)
GROUP BY year;
```

Implementation reality. Modern OLAP is mostly **ROLAP** - the cube is virtual, expressed as SQL on a star schema using `GROUP BY ... ROLLUP/CUBE`. Materialized views accelerate the common roll-ups.

Find it in the slides: Lecture 12-V2, slides 32-42.

8.7 Data engineering: the iceberg

Definition. Data engineering is the work of getting data **clean, conformed, fresh, and queryable**. The slides describe the analytic stack as an iceberg:

```
+----------------+
| Dashboards/BI  |  <- 5%, what users see
+----------------+
| OLAP / SQL     |  <- 10%
+----------------+
| Star schemas / |
| warehouse      |  <- 20%
+----------------+
| ETL / ELT      |
| pipelines,     |  <- 65%, the bulk of the engineering
```

```
| ingestion,     |     work, hidden from the user
| quality, ops   |
+----------------+
```

Why it dominates the time. Real-world sources are heterogeneous, broken, late, and changing. Schema drift, time-zone bugs, late-arriving facts, and semantic mismatches consume more effort than the analysis itself.

Find it in the slides: Lecture 11, slides 62-66; Lecture 12-V2, slides 43-45.

8.8 MapReduce

Definition. MapReduce is a batch parallel-processing model with two user-defined functions:

- **`map(k1, v1) -> list of (k2, v2)`** - applied independently to each input record.
- **`reduce(k2, list of v2) -> list of (k3, v3)`** - applied to all values that share a key (after a system-managed **shuffle/sort** step).

```
input split 1 -+-> map -+
input split 2 -+-> map -+--> shuffle/sort by key --+-> reduce --> output
input split N -+-> map -+                          +-> reduce --> output
                                                    ...
```

- **Why blocks matter.** Files are split into fixed-size **blocks** (HDFS default 64-128 MB). Each block becomes the input to one map task that runs **on the node holding the block** (data locality), enabling massive parallelism.
- **Why streams alone are insufficient.** A naive line-by-line stream forces all records through one consumer; the block model is what lets the framework spread work across thousands of nodes.
- **Hive and Pig** sit on top of MapReduce: SQL-like (Hive) or dataflow-like (Pig) languages that compile to MR jobs. Modern stacks (Tez, Spark) replaced raw MR while preserving the SQL surface.

Canonical example: word count. Count occurrences of each word across many documents.

```
map(docId, text):                          reduce(word, list_of_counts):
  for word in text.split():                  emit (word, sum(list_of_counts))
    emit (word, 1)
```

For the input `"the cat sat on the mat"`:

- `map` emits `(the,1) (cat,1) (sat,1) (on,1) (the,1) (mat,1)`.
- The framework **shuffles**, grouping by key: `the -> [1,1]`, `cat -> [1]`, ...
- `reduce` emits `(the,2) (cat,1) (sat,1) (on,1) (mat,1)`.

The same code runs unchanged on a 1 GB or 1 PB corpus; the framework just spawns more map and reduce tasks across the cluster.

Find it in the slides: Lecture 11, slides 67-71; Lecture 12-V2, slides 46-49.

8.9 Algebraic operators and Spark RDDs

Definition. Spark generalizes MapReduce to a graph of **algebraic operators** (`map`, `filter`, `flatMap`, `groupByKey`, `reduceByKey`, `join`, ...) over **RDDs** (Resilient Distributed Datasets) and DataFrames.

- An **RDD** is an immutable, partitioned collection of records, plus the **lineage** (sequence of operations) that created it. Lineage is how Spark recomputes lost partitions instead of replicating them.
- **Lazy evaluation.** `map`, `filter`, `join`, etc. are **transformations** that build the lineage graph but compute nothing. Only **actions** (`collect`, `count`, `save`) trigger execution. The optimizer can fuse and reorder transformations.
- **DataFrame / Dataset** API adds a relational schema and columnar-style execution - very close to a SQL engine that runs in a distributed cluster. (Real Spark calls its optimizer "Catalyst," but the deck does not name it.)
- **PySpark** is the Python API. Lambdas are shipped to the cluster.

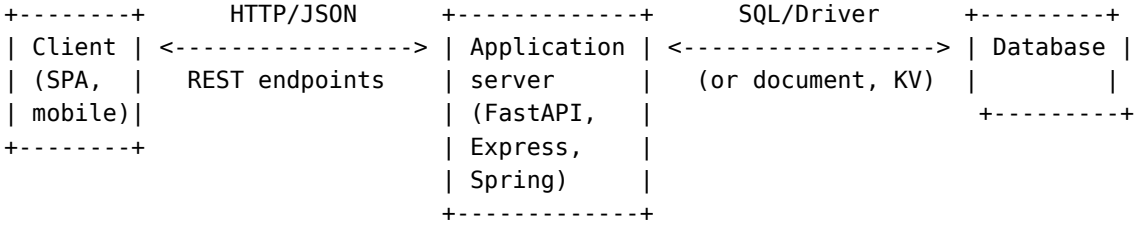
```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()

orders = spark.read.parquet("s3://bucket/orders/")
top = (orders
       .filter("status = 'PAID'")
       .groupBy("customer_id")
       .sum("amount")
       .orderBy("sum(amount)", ascending=False)
       .limit(10))
top.show()
```

Find it in the slides: Lecture 11, slides 72-77; Lecture 12-V2, slides 50-56.

Part 9: REST and Web Applications (Lecture 9 slides 102-113; Lecture 10 slides 80-92; Lecture 11 slides 78-90; Lecture 12-V2 slides 57-76)

9.1 Full-stack architecture

Definition. Modern web apps separate **presentation**, **application logic**, and **persistence** into independently scaled tiers.



- The application server is **stateless** at the request level (state lives in the DB or a session store).

- The **MERN** stack (Mongo, Express, React, Node) is the canonical example; the course emphasis is FastAPI + MySQL/Mongo.

Find it in the slides: Lecture 9, slides 102-104.

9.2 Web framework concepts: routes, models, OpenAPI

Definition. A **web framework** provides routing, request parsing, response serialization, and middleware so application code can focus on business logic.

- **Router.** Maps `(HTTP verb, URL pattern)` to a handler function.
- **Model.** A typed schema describing a request body or response (Pydantic in FastAPI; Mongoose in Node; SQLAlchemy ORM model). Enforces validation at the framework boundary.
- **OpenAPI.** A formal specification (in YAML/JSON) of every endpoint: paths, methods, parameters, request/response schemas, status codes. FastAPI generates OpenAPI automatically from the route signatures, and tools (Swagger UI, codegen) consume it.

Find it in the slides: Lecture 9, slide 105; Lecture 12-V2, slides 60-63.

9.3 The six REST constraints

#	Constraint	What it forces
1	**Client-server**	Clean separation of UI and storage. Independent evolution.
2	**Stateless**	The server keeps no per-client session state; each request carries everything needed (auth tokens, pagination cursors). Simplifies scaling and failover.
3	**Cacheable**	Responses must indicate whether they are cacheable (`Cache-Control`, `ETag`). Caches at the client / CDN reduce load.
4	**Uniform interface**	A small fixed set of operations on **resources** identified by URLs and acted on by **HTTP verbs**.
5	**Layered system**	The client cannot tell whether it is talking to the origin server, a load balancer, a CDN, or a gateway. Each layer can add cross-cutting concerns.
6	**Code-on-demand** (optional)	The server may ship executable code (JS) the client runs. The only optional constraint.

Find it in the slides: Lecture 9, slides 106-107; Lecture 10, slides 83-86; Lecture 12-V2, slides 64-66.

9.4 Resources and collection resources

- A **resource** is anything addressable by a URL: a customer, an order, a search result, a process.
- A **collection resource** is a resource whose representation is a list of other resources, typically `/customers`, `/orders`. Operating on the collection (`POST /customers`) creates a new member; operating on a member (`GET /customers/42`) acts on that one.
- **Hierarchical paths** express containment / one-to-many relationships:

```
/courses                        -- collection of courses
/courses/W4111                  -- one course
/courses/W4111/sections         -- collection of sections within W4111
/courses/W4111/sections/2       -- one section
/courses/W4111/sections/2/participants
                                -- collection of enrolled students in section 2
```

Find it in the slides: Lecture 9, slide 109; Lecture 12-V2, slides 67-72.

9.5 HTTP verbs and CRUD mapping

Important - what the lecture actually says. Lecture 9 slide 109, Lecture 10 slide 87, and Lecture 12-V2 slide 65 all state verbatim:

> "REST only allows four methods: POST (Create), GET (Retrieve), PUT (Update), DELETE (Delete). That's it. That's all you get."

If a written question asks "what HTTP methods does REST define?" or "give the CRUD-to-HTTP mapping," **answer with those four**. PATCH is included in the table below for completeness because it is part of the HTTP spec and used in real APIs, but it is **not** in Prof. Ferguson's slide deck.

Verb	CRUD	Idempotent?	Safe?	On collection `/orders`	On member `/orders/42`
`GET`	Read	yes	yes	List	Retrieve one
`POST`	Create	no	no	Create new (server assigns ID)	Often used for actions
`PUT`	Update / Replace	yes	no	Replace whole collection (rare)	Replace one (full body)
`PATCH`	Partial update *(not in lecture deck)*	no	no	(rare)	Update some fields
`DELETE`	Delete	yes	no	Delete whole collection (rare)	Delete one

- **Safe** = no side effect on resources.
- **Idempotent** = repeating the same request has the same effect as one request.
- **Status codes (general HTTP knowledge, not directly in the deck).** `200 OK`, `201 Created`, `204 No Content`, `400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`, `409 Conflict`, `422 Unprocessable Entity`, `500 Server Error`.

Concrete request/response examples (an `/orders` API).

```
POST /orders                                  -> 201 Created
  Body: {"customer":42, "amount":99}             Location: /orders/77
                                                 Body: {"id":77, ...}

GET  /orders/77                               -> 200 OK
                                                 Body: {"id":77, "customer":42,
"amount":99}

PUT  /orders/77                               -> 200 OK   (full replacement;
idempotent)
  Body: {"customer":42, "amount":120}

PATCH /orders/77                              -> 200 OK   (partial; not in lecture)
  Body: {"amount":120}

DELETE /orders/77                             -> 204 No Content   (idempotent)
DELETE /orders/77                             -> 404 Not Found    (still "the
resource is gone")
```

Notice that `POST` is **not idempotent**: two `POST /orders` requests create two orders (`/orders/77` and `/orders/78`). `PUT /orders/77` twice with the same body produces the same final state.

Find it in the slides: Lecture 9, slides 108-110; Lecture 10, slide 87; Lecture 12-V2, slides 65-67.

9.6 URLs, content types, and query parameters

- The path identifies the resource; query parameters refine the representation: `GET /customers?country=US&segment=PRO&page=2`.
- The query parameters typically translate directly into **SQL `WHERE`** clauses or MongoDB filters in the data service.
- Content negotiation via `Accept` header and `Content-Type` of the response: `application/json` is the default for REST APIs.
- For collections, **standardize pagination** (`?page=`, `?limit=`, `?after=`) and **sorting** (`?sort=name,-createdAt`).

Concrete translation example.

```
GET /customers?country=France&city=Paris&page=2&limit=50
```

```sql
SELECT *
FROM   customers
WHERE  country = 'France'
  AND  city    = 'Paris'
ORDER  BY customer_id
LIMIT  50 OFFSET 50;        -- page 2, 50 per page
```

The route handler reads the query string, the resource layer adds authorization checks, the data service builds and parameterizes the SQL, and the result rows are serialized as JSON. The same query parameters could also be translated into a Mongo filter `{country:"France", city:"Paris"}` with `.skip(50).limit(50)`.

Find it in the slides: Lecture 12-V2, slides 67-72.

9.7 Mapping CRUD/data model to REST

For each entity type that should be exposed over the API:

1. Identify the **primary key** - this becomes the path segment for member resources.
2. Choose a noun (plural) for the collection: `Customer` -> `/customers`.
3. Implement the five operations as the matching verbs.
4. Identify natural **subresources** (one-to-many) and nest them: `/customers/42/orders`.
5. For many-to-many, expose the relation as its own collection: `/enrollments` with members like `/enrollments/{student_id}/{section_id}`.

Sample-question pattern (Course / Section / Participant). This is the layout Prof. Ferguson uses in **Lecture 6 slide 37** and **Lecture 9 slide 110** - flat top-level collections plus *relative navigation paths*. Match this style on the exam:

```
-- Top-level collections and members:
GET, POST   /courses
GET, PUT, DELETE   /courses/<id>
GET, POST   /sections
GET, PUT, DELETE   /sections/<id>
GET, POST   /participants
GET, PUT, DELETE   /participants/<id>

-- Relative navigation paths (one-to-many / containment):
GET   /courses/<id>/sections          -- all sections of a course
GET   /sections/<id>/participants     -- all participants in a section
GET   /participants/<id>/sections     -- all sections a participant is in
```

GET on a collection (`/courses`, `/sections`, etc.) may also take query parameters that the data service translates into SQL `WHERE` predicates (e.g., `GET /customers?country=France&city=Paris`).

The difference between a **resource** and a **collection resource** in this model: a section is a single resource (`/sections/42`); the set of sections of a course is a collection resource (`/courses/W4111/sections`) that lists members or accepts new ones via POST.

9.8 Layered application pattern

The course's reference layering (FastAPI + MySQL):

```
+-----------------+
| Routes / Router |  <-- HTTP verbs, URLs, request validation
+-----------------+
| Resource layer  |  <-- business logic, authorization, orchestration
+-----------------+
| Data Service    |  <-- e.g., MySQLDataService: pure data access
| (DAO)           |
+-----------------+
| SQL / DB driver |  <-- parameterized SQL, transactions
+-----------------+
```

- **Why layer.** Each layer has one reason to change: the route layer changes when the API surface changes; the data service changes when the storage changes; the resource layer changes when business rules change. Tests can mock each boundary.
- **Testing.** The Python `requests` library is the standard for black-box testing of the HTTP surface; pytest + an in-memory data service for unit tests of the resource layer.

Find it in the slides: Lecture 12-V2, slides 73-75.

Appendix A: Slide cross-reference (topic -> deck/slides)

Use this table to jump back to the original lecture deck. "Slide N" = PDF page N for these decks.

Topic	Lecture deck	Slide range
Anomalies, redundancy	Lecture 6	49-51
Functional dependencies	Lecture 6	56-58
Closure / attribute closure	Lecture 6	59-61
Decomposition (lossless / dep-pres)	Lecture 6	53-55
BCNF	Lecture 6	63-64
3NF	Lecture 6	66-67
BCNF vs 3NF	Lecture 6	68
1NF-5NF summary	Lecture 6	74
Denormalization, materialized views	Lecture 6	72, 75-76
Compilation pipeline / EXPLAIN	Lecture 9	9-16
Selection algorithms (A1-A10)	Lecture 9	19-26
Join algorithms (NL, BNL, INL, merge, hash)	Lecture 9	28-41
Duplicate elimination, projection, aggregation	Lecture 9	42-44
Materialization vs pipelining; iterators	Lecture 9	45-51
Equivalence rules / push-down	Lecture 9, Lecture 10	57-73 / 13-25
Join order DP / `findbestplan`	Lecture 9, Lecture 10	74-77 / 26-30
Cost estimation, selectivity	Lecture 9, Lecture 10	76 / 31-33
ACID, A->B transfer	Lecture 10	34-43
Why naive commit fails	Lecture 10	44-49
WAL, log manager, steal/no-force	Lecture 10	50-53
ARIES (analysis/redo/undo)	Lecture 10	54
Durability + replication	Lecture 10	55-56
Serial vs concurrent schedules	Lecture 10, Lecture 11	57-60 / 12-23
Conflict serializability, precedence graph	Lecture 11	18-33
[illegible]	[illegible]	[illegible]

Appendix B: 60-second cheat summaries

Scan these in the last minutes before the exam.

B.1 Normalization

- FD `X -> Y`: same `X` always implies same `Y` on every legal instance.
- `X+` = closure. `X` is a superkey iff `X+ = R`.
- BCNF: every non-trivial FD has a superkey LHS. Always lossless. Not always dependency-preserving.
- 3NF: BCNF, OR `Y - X` is all prime. Always lossless and always dependency-preserving.
- Decomposition `R1, R2` is lossless iff `R1 INTERSECT R2` is a superkey of `R1` or `R2`.

B.2 Joins

- Hash join: equi-join, no index, both fit (or partition). Build smaller, probe larger.
- Indexed NL: small outer + indexed inner on join attr.
- Merge join: both sorted on join attr.
- Block NL: brute-force fallback. Outer = smaller relation.

B.3 Materialization vs pipelining

- Materialization: temp tables between operators; works for any operator; high latency.
- Pipelining: tuple-at-a-time via iterators; low latency; blocked by sort/hash-build.
- Iterator API: `open`, `next`, `close`.

B.4 ACID

- A: all-or-nothing.
- C: valid -> valid.
- I: as-if serial.
- D: survives crash.

B.5 WAL + ARIES

- Log records before data pages; force log on commit.
- Recovery: Analysis -> Redo (every logged action) -> Undo (active txns, in reverse, with CLRs).

B.6 Strict 2PL

- S/X locks; once you release any, you cannot acquire any; hold all locks until commit/abort.
- Guarantees conflict-serializable, recoverable, cascadeless, strict. Can deadlock.

B.7 Isolation levels

`P` = anomaly **possible** at this level. `-` = **prevented**.

Level	Dirty	Non-repeatable	Phantom
READ UNCOMMITTED	P	P	P
READ COMMITTED	-	P	P
REPEATABLE READ	-	-	P (lecture: "range locks are not managed"; real MySQL InnoDB usually prevents it via next-key/gap locks)
SERIALIZABLE	-	-	-

Snapshot isolation (Oracle, older Postgres) prevents all three but allows **write skew**.

B.8 Deadlock

- Detect: wait-for graph cycle.
- Prevent: ordering, timeouts, **wait-die** (older waits, younger dies), **wound-wait** (older wounds, younger waits).
- Recover: pick victim, roll back; avoid starvation.

B.9 BASE / CAP

- BASE = Basically Available, Soft state, Eventually consistent.
- CAP = pick 2 of {Consistency, Availability, Partition tolerance}; partitions are inevitable, so the real choice is **CP vs AP**.

B.10 Scale-out

- Shared-nothing, sharded by key. Easy when workload partitions cleanly; hard joins, hard distributed transactions.

B.11 NoSQL

- Document (Mongo), key-value (Dynamo, Redis), wide-column (Cassandra), graph (Neo4j).
- Mongo: db -> collection -> document -> field; `find(filter, projection)`; aggregation pipeline.

B.12 Big data

- 5 V's: Volume, Velocity, Variety, Veracity, Value.
- Warehouse = schema on write; lake = schema on read; lakehouse = lake + table layer.
- ETL = transform before load; ELT = transform inside the warehouse.

B.13 Star schema and OLAP

- Fact = events with measures + FKs to dimensions.
- Dimensions = descriptive context; have hierarchies.

- Operators: slice, dice, roll-up, drill-down, pivot.

B.14 MapReduce + Spark

- map -> shuffle/sort -> reduce; data locality on HDFS blocks.
- Spark RDDs: lazy transformations + actions; lineage gives fault tolerance.

B.15 REST

- Six constraints: client-server, stateless, cacheable, uniform interface, layered, code-on-demand.
- Resources by URL, actions by verbs. **Lecture says four:** GET / POST / PUT / DELETE. (PATCH is real-world but not in the deck.)
- Collection resource (`/orders`) vs member (`/orders/42`); subresources by nesting (`/courses/<id>/sections`) or as their own top-level collection (`/sections`).
- Layered app: routes -> resource -> data service -> SQL.